

Universidad ORT - Uruguay

Facultad de Ingeniería

Ingeniería en sistemas

Taller de tecnologías

Obligatorio 1 - Gestor del Mundial

- Renzo Amestoy - 361801
- Santiago Rodríguez - 376414

Clase: M1C

Docente: Natalia Franco

Fecha de entrega: 20/5/2026

Indice:

Sección 1 - Encabezado y Función "listaEquipos".....Pág. 3

Sección 2 - Función
"campeon".....Pág. 4

Sección 3 - Función
"registrarEquipo".....Pág. 4

Sección 4 - Función
"registrarPartido".....Pág. 5

Sección 5 - Función
"historial".....Pág. 6

Sección 6 - Función "buscarEquipo"Pág. 7

Sección 7 - Función "cantidadPartidos"Pág. 7

Sección 8 - Función "salir" y menú principal
.....Pág. 8

El siguiente programa fue desarrollado en Bash y tiene como objetivo gestionar información relacionada a un Mundial de fútbol, permitiendo registrar equipos, partidos y consultar distintos datos mediante un menú interactivo.

Sección 1 - Encabezado y Función “ listaEquipos ”

```
CAMPEON=Argentina  
  
EQUIPO=Argentina  
EQUIPO=Francia  
EQUIPO=Brasil  
EQUIPO=Uruguay
```

archivo “información”

```
#!/bin/bash  
  
#Renzo Amestoy - 361801 / Santiago Rodriguez - 376414  
  
ARCHIVO="informacion"  
  
listaEquipos() {  
    echo "-----"  
    grep "EQUIPO" $ARCHIVO | cut -d "=" -f 2  
    echo "-----"  
}
```

Comenzamos el script con “#!/bin/bash”, que es la línea necesaria para indicarle al sistema operativo que el archivo debe ser interpretado con el intérprete de Bash. A continuación, agregamos como comentario los nombres y números de estudiante de ambos integrantes del grupo.

Luego, definimos la variable **ARCHIVO** asignándole el valor “informacion”, que corresponde al archivo de texto en el cual se almacena toda la información predeterminada del programa: los equipos registrados y el campeón actual.

Continuando con la primera función que desarrollamos, la cual fue **listaEquipos()**. Su objetivo es mostrar en pantalla todos los equipos que han sido registrados. Para ello, utilizamos el comando **grep "EQUIPO" \$ARCHIVO**, que busca dentro del archivo todas las líneas que contienen la palabra EQUIPO en mayúsculas, ya que así es exactamente como se

almacenan los datos en el archivo información. Luego, a través de una tubería/pipe '|', le pasamos el resultado al comando `cut -d "=" -f 2`, que se encarga de cortar el texto usando el signo = como delimitador y quedarse únicamente con lo que está a la derecha del mismo, es decir, el nombre del equipo. Para mejorar la presentación visual de la salida, agregamos líneas decorativas con echo tanto arriba como abajo del resultado.

Sección 2 - Función `campeon`

```
campeon() {  
    echo "-----"  
    grep "CAMPEON" $ARCHIVO | cut -d "=" -f 2  
    echo "-----"  
}
```

Esta función sigue exactamente la misma lógica que `listaEquipos()`, con la única diferencia de que en lugar de buscar la palabra EQUIPO, se busca la palabra CAMPEON dentro del archivo. De esta forma, muestra el nombre del campeón actual registrado. Manteniendo el mismo estilo con las líneas decorativas de echo.

Sección 3 - Función `registrarEquipo`

```
registrarEquipo() {  
    equipos=$(grep "EQUIPO" $ARCHIVO | wc -l)  
    if [ "$equipos" -ge 15 ]; then  
        echo "-----"  
        echo "El limite es 15 equipos"  
        echo "-----"  
        return  
    fi  
    echo "-----"  
    echo "Ingrese un nuevo equipo: "  
    echo "-----"  
    read nombreEquipo  
    if [ -z "$nombreEquipo" ]; then  
        echo "-----"  
        echo "El nombre no puede estar vacio"  
        echo "-----"  
        return  
    fi  
    echo "-----"  
    echo "EQUIPO=$nombreEquipo" >> $ARCHIVO  
    echo "-----"  
}
```

Esta función incorpora los conceptos básicos de las anteriores y agrega lógica de validación. Lo primero que hacemos es definir una variable llamada **equipos**, a la cual le asignamos el resultado de buscar con **grep "EQUIPO"** dentro del archivo y pasarlo por **wc -l**. El comando **wc -l**, cuenta la cantidad de líneas que devuelve **grep**, lo que equivale a contar cuántos equipos hay registrados en ese momento. Con ese número calculado, evaluamos mediante una condición **if** si la cantidad de equipos es mayor o igual a 15. En caso de serlo, la función muestra un mensaje indicando que se alcanzó el límite máximo de equipos y retorna sin realizar ninguna acción adicional, impidiendo el registro de nuevos equipos. Si la cantidad es menor a 15, el programa solicita al usuario que ingrese el nombre del nuevo equipo, leyendo la respuesta con **read** y almacenándola en la variable **nombreEquipo**. Luego se verifica que dicha variable no esté vacía mediante la condición **[-z "\$nombreEquipo"]**; si está vacía, se informa al usuario que el nombre no puede ser un campo vacío y se retorna. Si el nombre es válido, se guarda en el archivo de información con el formato **EQUIPO=\$nombreEquipo**, utilizando **">>"** para agregar la línea al final del archivo sin sobrescribir el contenido existente.

Sección 4 - Función registrarPartido

```

registrarPartido() {
    echo "-----"
    echo "Ingrese equipo 1:"
    echo "-----"
    read equipo1
    if [ -z "$equipo1" ]; then
        echo "-----"
        echo "El nombre no puede estar vacio"
        echo "-----"
        return

    fi
    echo "-----"
    echo "Ingrese equipo 2:"
    echo "-----"
    read equipo2

    if [ -z "$equipo2" ]; then
        echo "-----"
        echo "El nombre no puede estar vacio"
        echo "-----"
        return

    fi
    echo "-----"
    echo "Ingrese goles equipo 1:"
    echo "-----"
    read goles1
    if [ -z "$goles1" ]; then
        echo "-----"
        echo "El campo no puede estar vacio"
        echo "-----"
        return

    fi
    echo "-----"
    echo "Ingrese goles equipo 2:"
    echo "-----"
    read goles2
    if [ -z "$goles2" ]; then
        echo "-----"
        echo "El campo no puede estar vacio"
        echo "-----"
        return

    fi
    echo "-----"
    echo "PARTIDO=$equipo1 $goles1 $equipo2 $goles2" >> $ARCHIVO
    echo "-----"
}

```

Para poder registrar un partido mediante esta función, primero solicita al usuario que ingrese el nombre del equipo 1, lo almacena en la variable **equipo1** y verifica que no esté vacío. Luego repite el mismo proceso para el equipo 2 con la variable **equipo2**.

A continuación, solicita la cantidad de goles de cada equipo en ese partido, almacenándolos en las variables **goles1** y **goles2** respectivamente, validando también que ninguno de esos campos quede vacío. Finalmente, una vez recopilados y validados todos los datos, se guarda el resultado del partido en el archivo de información con el formato **PARTIDO=\$equipo1 \$goles1 \$equipo2 \$goles2**, utilizando ">>" para agregarlo al final del archivo.

Sección 5 - Función historial

```

historial() {
    partidos=$(grep "PARTIDO" $ARCHIVO)

    if [ -z "$partidos" ]; then
        echo "-----"
        echo "No hay partidos registrados"
        echo "-----"
        return
    fi
    echo "-----"
    echo "$partidos" | sed 's/PARTIDO=/El resultado del partido fue: /'
    echo "-----"
}

```

Para esta función, primero definimos la variable **partidos** asignándole el resultado de buscar con **grep "PARTIDO"** dentro del archivo de información. Si **grep** no encuentra ninguna coincidencia, la variable quedará vacía, lo cual se evalúa con la condición **[-z "\$partidos"]**. En ese caso, la función muestra el mensaje "No hay partidos registrados" y retorna. Si en cambio la variable contiene datos, significa que hay partidos guardados, y la función los muestra reemplazando la palabra **PARTIDO=** por el texto "**El resultado del partido fue: "** mediante el comando **sed**, presentando así la información de forma más legible para el usuario.

Sección 6 - Función buscarEquipo

```

buscarEquipo() {
    echo "-----"
    echo "Ingrese un equipo para buscar:"
    echo "-----"
    read nombreEquipo
    if [ -z "$nombreEquipo" ]; then
        echo "-----"
        echo "El campo no puede estar vacio"
        echo "-----"
        return

    fi

    resultado=$(grep "EQUIPO=$nombreEquipo" $ARCHIVO)

    if [ -z "$resultado" ]; then
        echo "-----"
        echo "El equipo no existe"
        echo "-----"
    else
        echo "-----"
        echo "El equipo existe"
        echo "-----"
    fi
}

```

Esta función solicita al usuario que ingrese el nombre de un equipo, lo almacena en la variable **nombreEquipo** y verifica que no esté vacía. Luego realiza una búsqueda con **grep "EQUIPO=\$nombreEquipo"** dentro del archivo de información, almacenando el resultado en la variable **resultado**. Si la variable resultado está vacía, significa que grep no encontró ninguna coincidencia, por lo que el programa informa que el equipo no existe. En caso contrario, si se encontró una coincidencia, informa que el equipo sí existe. De esta forma, la función permite verificar rápidamente si un equipo determinado ha sido registrado previamente en el sistema.

Sección 7 - Función cantidadPartidos

```

cantidadPartidos() {
    echo "-----"
    echo "Cantidad de partidos jugados:"

    grep "PARTIDO" $ARCHIVO | wc -l
    echo "-----"
}

```

Esta función cuenta cuántos partidos hay registrados. Para ello, utiliza **grep "PARTIDO" \$ARCHIVO | wc -l**, que busca todas las líneas que contienen la palabra PARTIDO y las cuenta. El resultado se muestra en pantalla con el mensaje "Cantidad de partidos jugados: ", indicándole al usuario cuántos partidos han sido registrados hasta el momento.

Sección 8 - Función salir y menú principal

```

salir() {
    exit 0
}

while true
do
    echo "                                     === Bienvenido al mundial ==="
    echo "1-Lista de equipos"
    echo "2-Campeon actual"
    echo "3-Registrar Equipo"
    echo "4-Registrar Partido"
    echo "5-Historial"
    echo "6-Buscar equipo"
    echo "7-Cantidad de Partidos"
    echo "8-Salir"
    read opcion

    case $opcion in
        1) listaEquipos ;;
        2) campeon ;;
        3) registrarEquipo ;;
        4) registrarPartido ;;
        5) historial ;;
        6) buscarEquipo ;;
        7) cantidadPartidos ;;
        8) salir ;;
        *) echo "-----"
            echo "  Opcion invalida"
            echo "-----" ;;
    esac

done

```

La función salir es la más sencilla del programa: únicamente ejecuta **exit 0**. El comando **exit** finaliza la ejecución del script, y el valor **0** que se le pasa como argumento, es el código de salida, que se lo conoce por terminar un programa correctamente y sin errores. Si el código fuera distinto de cero, indicaría que hubo algún problema durante la ejecución.

Por último, el menú principal está implementado dentro de un bucle **while true**, lo que significa que el programa se ejecuta de forma continua e indefinida hasta que el usuario elija explícitamente la opción de salir. En cada iteración, se muestra el menú con un **echo** que incluye espacios y caracteres decorativos para que el título "**Bienvenido al mundial**" quede visualmente centrado y estéticamente agradable. A continuación se listan las ocho opciones disponibles y se lee la elección del usuario con **read**. Mediante una estructura **case**, el programa evalúa qué opción fue ingresada y llama a la función correspondiente. Si el usuario ingresa cualquier valor que no corresponda a una de las ocho opciones válidas, el caso **"*"** captura esa situación y muestra el mensaje "**Opción inválida**", volviendo a mostrar el menú en la siguiente iteración del bucle.